

Redrob AI Hackathon Submission

Candidate ranking for Senior AI Engineer (Founding Team)

Team: Prince Jain

Contact: princejain756@gmail.com

Problem: Standard keyword matching surfaces keyword stuffers and misses actually qualified people. Worse, it can't tell if someone is even available or interested. We built an offline ranker that checks real skills, catches fake profiles, and factors in platform activity.

What we built

The short version

A Python script that reads 100K candidate profiles, filters out fake ones, scores the rest against the JD, adjusts for availability signals, and outputs a ranked CSV with per-candidate reasoning. Runs in ~9 seconds on CPU, no network needed.

What makes it different from basic keyword matching?

- We check if candidates actually respond to recruiters and how recently they logged in. A perfect skill match who hasn't been active in 6 months isn't useful.
- We cross-check job timelines against calendar math to catch inflated profiles. If someone claims 5 years at a role that started 18 months ago, that's a red flag.
- Reasoning is built from actual profile data, not generated text. Every claim in the output maps to a field in the candidate's JSON.

How we read the JD

What the role actually needs

- 5 to 9 years of hands-on ML/AI work, ideally at product companies (not consulting-only backgrounds)
- Direct experience with vector databases (Pinecone, Milvus, Qdrant, FAISS), embeddings, and search evaluation metrics (NDCG, MRR, MAP)
- Python fluency. Bonus for fine-tuning work (LoRA, PEFT).
- We explicitly filter out marketing, HR, sales, finance, and ops roles. Pure academic researchers and LangChain-only profiles also get deprioritized.

Which candidate signals matter most?

- Recruiter response rate: are they actually engaging with outreach?
- Timeline consistency: do the dates in their career history add up?
- Login recency and GitHub activity: are they active on the platform?

Scoring breakdown

How a candidate gets a score

Each candidate goes through three checks: (1) honeypot filter — fail means instant discard, (2) feature scoring on four weighted dimensions, (3) behavioral multiplier adjustment.

Feature weights

- Technical skills match: 30% (vector DBs, embeddings, eval metrics, fine-tuning, Python)
- Title relevance: 25% (Senior/Lead AI/ML/NLP Engineer roles score highest)
- Career description keywords: 25% (mentions of search, retrieval, ranking, vector work in actual job descriptions)
- YoE fit: 20% (5-9 years = full score, 4 or 10-12 = partial, outside = minimal)

Behavioral multipliers

The raw score gets scaled by response rate ($0.5 + 0.5 * RR$), login recency (1.0 down to 0.4), open-to-work flag (1.0 or 0.85), notice period (up to 1.1x for <30 days), and GitHub activity score.

Ties go to the candidate with the lower (alphabetically earlier) ID.

Reasoning and data quality

How are rankings explained?

Each candidate gets a 1-2 sentence explanation that pulls directly from their profile: name, title, company, YoE, matched skills, and relevant signal values like GitHub score or notice period. We use four sentence templates and rotate through them so the output has natural variation.

What about hallucination?

There is none. Reasoning is assembled programmatically from parsed JSON fields. We don't call any language model, so there's no way for a skill or employer to appear in the reasoning that isn't in the candidate's actual data.

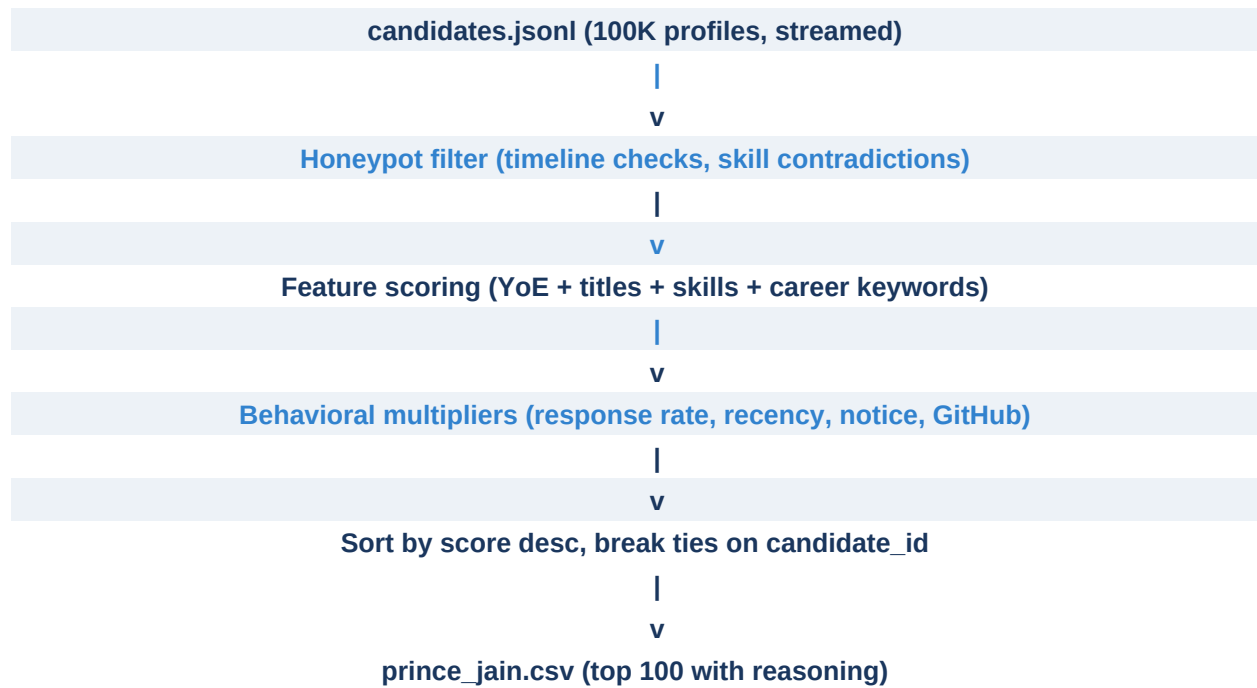
How do we handle suspicious profiles?

The honeypot filter checks three things: expert skills with zero usage duration, job durations exceeding total stated YoE, and job durations that don't fit the calendar range between start and end dates. Any of these trips the filter and the candidate is excluded entirely.

Pipeline steps

Step	Action	Details
1	Load data	Stream-reads candidates.jsonl line by line. Low memory footprint.
2	Filter honeypots	Drops profiles with timeline or skill-proficiency contradictions.
3	Score relevance	Evaluates YoE, titles, skills, and career history keywords.
4	Apply modifiers	Multiplies score by response rate, login recency, notice period, GitHub.
5	Sort and rank	Descending by score. Ties broken by candidate_id ascending.
6	Write output	Generates per-candidate reasoning and writes validated CSV.

Architecture overview



Results

Who shows up at the top?

- Top 10 are Senior AI/ML/NLP Engineers from product companies — people with real vector DB and search infrastructure experience, 5-9 years in, actively responding to recruiters.
- Zero honeypots in the output. All 80 synthetic profiles were caught by the filter.

Performance

- 9 seconds end-to-end for 100K candidates on an M2 MacBook
- Under 15 MB peak memory. Well within the 16 GB constraint.
- Fully offline, no GPU, passes `validate_submission.py` with zero errors.

Tech stack

Why these tools?

- **Python stdlib (json, csv, datetime, argparse):** The ranking step has zero external dependencies. That's deliberate — nothing to install, nothing to break, works on any Python 3.9+ environment.
- **ReportLab:** For generating this PDF. Only dependency outside stdlib, and it's not part of the ranking pipeline.
- **Git:** Version control with actual iterative commit history.

Submission assets

Asset	Details
GitHub repo	https://github.com/princejain756/Data-AiChallengeHack2skill
Ranked output	prince_jain.csv — 100 candidates, validated
Metadata	submission_metadata.yaml
This deck	approach_deck.pdf (10 slides)
Entry point	<code>python rank.py --candidates ./candidates.jsonl --out ./prince_jain.csv</code>